

COLLEGIUM WITELONA UCZELNIA PAŃSTWOWA

Wydział Nauk Technicznych i Ekonomicznych

DOKUMENTACJA PROJEKTU

System Wypożyczalni Książek i Księgarni Internetowej

Projektowanie systemów baz danych

Wykonawcy:

Kyrylo Shchedrin,

Nikola Pisarska

Semestr: IV

Rok akademicki: 2025/2026

Legnica, 2026

1. Opis struktury bazy danych.....	3
1.1. Tabele i ich struktura szczegółowa.....	3
1.2. Graficzny schemat relacyjny bazy danych (ERD).....	5
1.3. Zabezpieczenia spójności i powiązań (Fluent API).....	6
2. Główne funkcje systemu.....	7
2.1. Panel Administratora (Zarządzanie ofertą i specyfikacją).....	7
2.2. Autoryzacja i bezpieczeństwo (Kryptografia BCrypt).....	7
2.3. Moduł Sklepu Internetowego i Wypożyczeń.....	8
3. Sposób działania środowiska Docker.....	9
3.1. Architektura sieciowa środowiska (Docker Network).....	9
3.2. Zasada działania pliku Dockerfile (Multi-stage build).....	9
3.3. Izolacja i trwałość danych (Docker Volumes).....	9
4. Instrukcja uruchomienia projektu.....	10
5. Instrukcja wersjonowania i zarządzania kodem projektu.....	11
5.1. Podejście do kontroli wersji.....	11
5.2. Procedura wersjonowania na potrzeby przyszłego rozwoju.....	11

1. Opis struktury bazy danych

Baza danych systemu została zaprojektowana w relacyjnym modelu danych i zrealizowana przy użyciu serwera MySQL w wersji 8.0.30. Mapowanie obiektowo-relacyjne (ORM) oraz automatyczna generacja schematu zostały wykonane za pomocą narzędzia Entity Framework Core w podejściu Code-First.

1.1. Tabele i ich struktura szczegółowa

Poniższy opis przedstawia przeznaczenie oraz logikę biznesową poszczególnych encji systemu bazodanowego. Pełny graficzny model relacyjny (ERD), zawierający szczegółowe typy danych (takie jak **INT**, **LONGTEXT**, **DECIMAL**), oznaczenia kluczy głównych (PK), obcych (FK) oraz opcjonalności pól (NULL/NOT NULL), został zamieszczony w postaci zbiorczego diagramu na końcu tej sekcji.

Tabela: Books (Książki / Produkty) Przechowuje pełne informacje handlowe, marketingowe oraz zaawansowaną specyfikację techniczną i wydawniczą publikacji literackich. Stanowi centralny katalog produktów wspólny dla modułu darmowych wypożyczeń bibliotecznych oraz modułu komercyjnej sprzedaży w księgarni internetowej. Zawiera m.in. szczegółowe wymiary fizyczne wolumenu (wysokość, głębokość, szerokość w milimetrach), kody identyfikacyjne (EAN/ISBN, wewnętrzny indeks magazynowy) oraz dane dotyczące certyfikacji bezpieczeństwa produktu (GpsrCertyfikaty).

Tabela: Egzemplarze (Fizyczne sztuki utworów) Reprezentuje konkretne, fizyczne sztuki przypisane do danej pozycji książkowej w module bibliotecznym, umożliwiając precyzyjne śledzenie stanu każdego wolumenu. Każdy egzemplarz posiada unikalny numer inwentarzowy generowany systemowo na podstawie algorytmu GUID oraz przypisany status stanu technicznego (np. idealny, zużyty, zniszczony). Tabela jest powiązana z tabelą **Books** relacją klucza obcego z włączonym usuwaniem kaskadowym.

Tabela: Klienci (Dane osobowe konsumentów) Przechowuje profile osobowe oraz unikalne dane kontaktowe (imię, nazwisko, adres e-mail, numer telefonu) klientów korzystających z usług biblioteki oraz sklepu internetowego. Stanowi warstwę informacyjną o konsumencie, odizolowaną od danych uwierzytelniających.

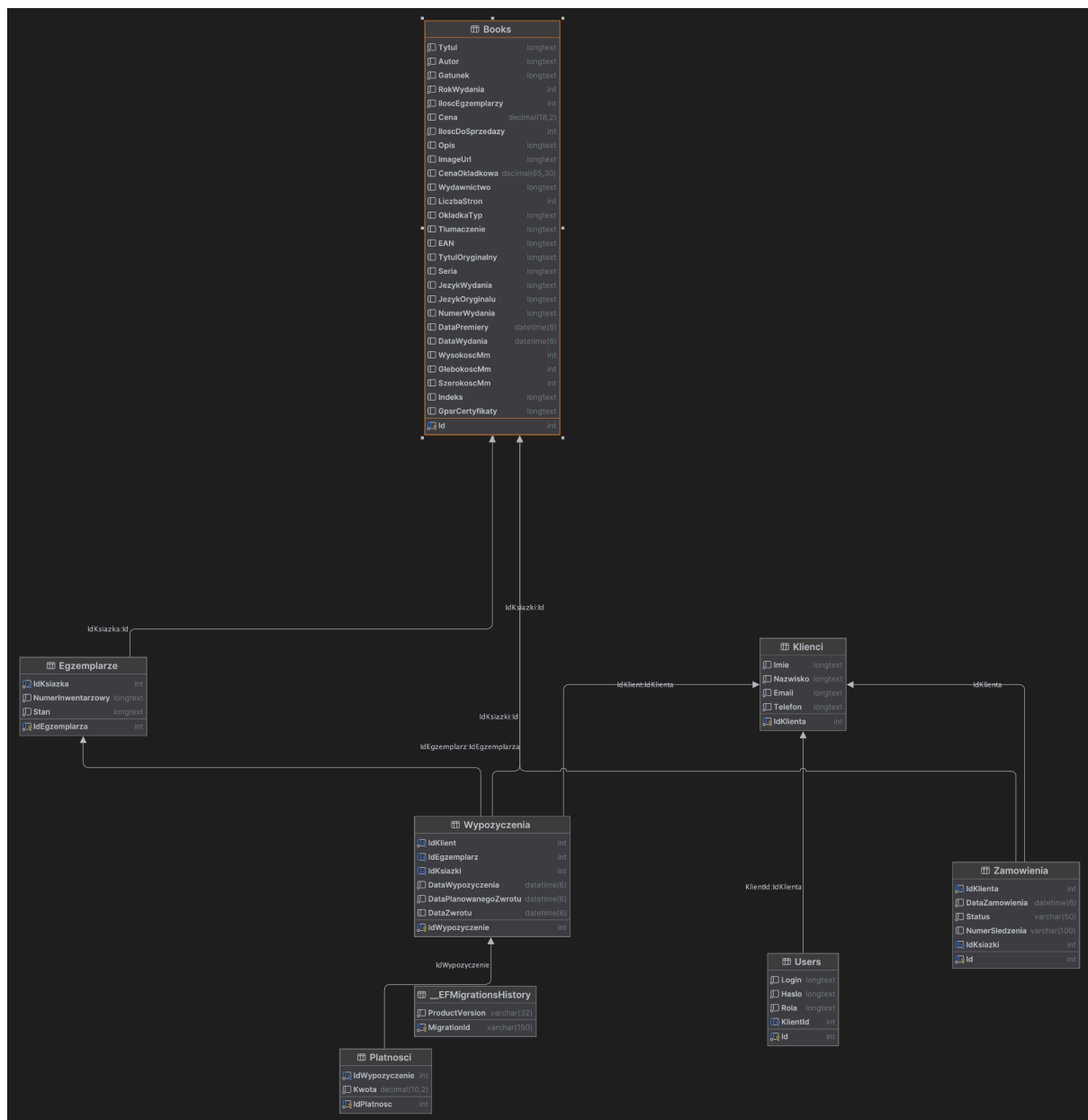
Tabela: Users (Konta uwierzytelniające w systemie) Odpowiada za bezpieczeństwo i kontrolę dostępu do systemu. Przechowuje unikalne loginy, przypisane role systemowe (Admin, Klient) oraz zabezpieczone skróty kryptograficzne haseł wygenerowane za pomocą jednokierunkowego algorytmu haszującego BCrypt. Tabela jest powiązana kluczem obcym **KlientId** z tabelą **Klienci** z restrykcją **ON DELETE SET NULL**, co chroni konto przed usunięciem w przypadku anonimizacji danych osobowych klienta.

Tabela: Wypożyczenia (Transakcje biblioteczne) Rejestruje pełny cykl życia operacji bibliotecznych realizowanych przez klientów. Przechowuje dokładne znaczniki czasu zawarcia transakcji, planowanego terminu zwrotu oraz faktycznej daty oddania wolumenu. Łączy relacjami kluczy obcych profil klienta, tabelę katalogową **Books** oraz konkretny fizyczny **Egzemplarz**.

Tabela: Zamowienia (Transakcje komercyjne sklepu) Ewidencjonuje zamówienia zakupowe składane przez klientów w module księgarni internetowej. Przechowuje datę złożenia zamówienia, aktualny status jego realizacji (np. Nowe, Opłacone, Wysłane) oraz numer listu przewozowego przesyłki kurierskiej. Jest powiązana kluczami obcymi z tabelą **Klienci** oraz bezpośrednio z katalogiem **Books** (sprzedaż odbywa się z puli egzemplarzy przeznaczonych do handlu detalicznego).

Tabela: Płatności (Rozliczenia finansowe) Służy do rejestrowania i ewidencjonowania wszystkich opłat wnoszonych przez użytkowników. Przechowuje wartość transakcji finansowej (Kwota) o jawnej precyzji numerycznej. Posiada opcjonalne powiązanie klucza obcego z tabelą **Wypożyczenia**, co pozwala na jednoznaczne identyfikowanie i rozliczanie kar regulaminowych naliczonych za przetrzymanie zasobów bibliotecznych.

1.2. Graficzny schemat relacyjny bazy danych (ERD)



1.3. Zabezpieczenia spójności i powiązań (Fluent API)

W klasie konfiguracji kontekstu `LibraryDbContext` zaimplementowano rygorystyczne reguły dbające o integralność referencyjną oraz spójność danych bezpośrednio na poziomie silnika bazodanowego MySQL:

Precyzja pól numerycznych: Dla pól przechowujących wartości finansowe (`Book.Cena` oraz `Platnosc.Kwota`) wymuszono jawną precyzję stałoprzecinkową za pomocą metody Fluent API `HasPrecision()`. Wykorzystanie typu danych `DECIMAL` zamiast typów przybliżonych (takich jak `FLOAT` czy `DOUBLE`) całkowicie zapobiega utracie groszy wynikającej z błędów zaokrągleń matematycznych podczas operacji na bazie danych.

Restrykcja usuwania (`DeleteBehavior.Restrict`): Powiązanie klucza obcego między tabelą wypożyczeń (`Wypozyczenia`) a katalogiem książek (`Books`) zostało zabezpieczone przed usunięciem rekordu nadrzędnego. Jeśli dana książka posiada jakiegokolwiek aktywny lub historyczny wpis o wypożyczeniu, system zablokuje jej usunięcie, chroniąc bazę przed powstawaniem tzw. rekordów osieroconych.

Ustawianie pustej wartości (`DeleteBehavior.SetNull`): W przypadku podjęcia decyzji o usunięciu profilu osobowego konsumenta z tabeli `Klienci`, powiązany z nim rekord uwierzytelniający w tabeli `Users` nie jest usuwany kaskadowo. Klucz obcy `KlientId` przyjmuje wówczas stan `NULL`. Zapobiega to utracie konta systemowego i pozwala zachować pełną spójność oraz ciągłość operacyjną logów bezpieczeństwa.

2. Główne funkcje systemu

Aplikacja realizuje hybrydowy model biznesowy łączący zadania tradycyjnej biblioteki publicznej z modułem komercyjnej księgarni internetowej oraz panelem analityczno-zarządczym dla personelu.

2.1. Panel Administratora (Zarządzanie ofertą i specyfikacją)

Główny punkt sterowania systemem, dostępny wyłącznie dla użytkowników posiadających autoryzację typu `Admin`. Panel umożliwia pełne zarządzanie zasobami poprzez operacje CRUD (Create, Read, Update, Delete) w ramach następujących funkcjonalności:

Kreator nowych produktów: Pozwala na jednoczesne wprowadzenie pełnych parametrów książki (w tym bogatych danych wymiarowych i wydawniczych) oraz automatyczne zapisanie przesłanej grafiki okładki w katalogu serwera. W trakcie tego procesu określana jest liczba sztuk przeznaczona do sprzedaży detalicznej oraz automatycznie generowane są fizyczne, unikalne egzemplarze do modułu wypożyczeń bibliotecznych (systemowa pętla tworzy unikalne numery inwentarzowe na podstawie algorytmu opartego o identyfikatory GUID).

Zarządzanie stanem technicznym: Umożliwia personelowi bieżącą aktualizację oceny zużycia każdej fizycznej sztuki publikacji (dostępne statusy: idealny, zużyty, zniszczony).

Transakcyjne bezpieczne usuwanie: Proces usuwania zniszczonego egzemplarza został zabezpieczony transakcyjnie. System automatycznie weryfikuje, czy dany wolumen nie jest aktualnie przypisany do aktywnego wypożyczenia przez klienta, co chroni bazę danych przed powstawaniem błędów logicznych i przerwaniem spójności referencyjnej.

2.2. Autoryzacja i bezpieczeństwo (Kryptografia BCrypt)

System kładzie wysoki nacisk na poufność i bezpieczeństwo danych uwierzytelniających użytkowników:

Haszowanie haseł: System kategorycznie nie przechowuje haseł użytkowników w postaci otwartego tekstu. Podczas rejestracji nowego konta lub automatycznej inicjalizacji (seedowania) systemu, hasło przechodzi przez zaawansowany, jednokierunkowy algorytm haszujący z automatycznym generowaniem soli kryptograficznej za pomocą metody `BCrypt.Net.BCrypt.HashPassword(password)`.

Weryfikacja tożsamości: Podczas procesu logowania w metodzie kontrolera następuje bezpieczne, odporne na ataki typu *timing attack* porównanie ciągu przesłanego przez użytkownika z haszem pobranym z bazy danych za pomocą metody `BCrypt.Verify`.

2.3. Moduł Sklepu Internetowego i Wypożyczeń

Obsługa Koszyka i stanów sesji: Wybory zakupowe i biblioteczne klientów są odizolowane i zabezpieczone dzięki wykorzystaniu wbudowanego silnika sesji ASP.NET Core (`HttpContext.Session`). Zastosowanie flagi `HttpOnly` na ciasteczkach sesyjnych oraz mechanizmu `HttpContextAccessor` gwarantuje wysoki poziom bezpieczeństwa i uniemożliwia dostęp do tokenów sesji z poziomu skryptów klienckich.

Zarządzanie wypożyczeniami i zamówieniami: System automatycznie weryfikuje dostępność wolnych egzemplarzy fizycznych przed zatwierdzeniem wypożyczenia. Ponadto moduł odpowiada za rejestrację terminów, obsługę procesów sprzedażowych (zarządzanie statusem zamówienia komercyjnego) oraz wyliczanie wskaźników analitycznych (takich jak rejestr aktywnych dłużników czy automatyczne naliczanie nieopłaconych kar regulaminowych).

3. Sposób działania środowiska Docker

Wdrożenie kontenerowe w projekcie opiera się o architekturę wielokontenerową, sterowaną i koordynowaną przy użyciu narzędzia **Docker Compose**. Całe środowisko uruchomieniowe (zarówno warstwa aplikacyjna, jak i silnik bazy danych) zostało całkowicie odizolowane i uniezależnione od systemu operacyjnego oraz lokalnych konfiguracji komputera hosta, co skutecznie eliminuje powszechny problem deweloperski typu „u mnie nie działa”.

3.1. Architektura sieciowa środowiska (Docker Network)

Kontenery komunikują się ze sobą wewnątrz odizolowanej od świata zewnętrznego sieci mostkowej o nazwie `meow_network`. Wykorzystanie wbudowanego w silnik Dockera wewnętrznego serwera DNS pozwala na dynamiczne adresowanie usług. Dzięki temu aplikacja webowa (`meow_app`) łączy się z bazą danych bez konieczności wpisywania sztywnych adresów IP, posługując się bezpośrednio nazwą usługi serwera zdefiniowaną w pliku `docker-compose.yml` (parametr połączenia: `server=meow_db`).

3.2. Zasada działania pliku Dockerfile (Multi-stage build)

Plik `Dockerfile` wykorzystuje zaawansowany i zoptymalizowany proces budowania wieloetapowego (*Multi-stage build*). Pozwala on na drastyczne zmniejszenie rozmiaru finalnego obrazu kontenera oraz uniemożliwia wyciek kodu źródłowego na środowisko produkcyjne:

1. **Etap budowania (SDK - Software Development Kit):** W pierwszym kroku pobierany jest pełny obraz deweloperski `mcr.microsoft.com/dotnet/sdk:10.0` zawierający kompilator języka C# oraz narzędzia menedżera pakietów NuGet. Na tym etapie następuje pobranie i przywrócenie wszystkich niezbędnych bibliotek (`dotnet restore`) oraz pełna kompilacja kodu źródłowego aplikacji do zoptymalizowanych plików binarnych (`dotnet publish -c Release`).
2. **Etap uruchomieniowy (Runtime - ASP.NET Core):** Gotowe, skompilowane pliki binarne oraz biblioteki `.dll` są kopiowane z pierwszego etapu do drugiego, ultra-lekkiego obrazu bazowego `mcr.microsoft.com/dotnet/aspnet:10.0`. Obraz ten pozbawiony jest kompilatora i ciężkich narzędzi SDK, a zawiera jedynie lekkie środowisko uruchomieniowe oraz serwer webowy Kestrel. Gwarantuje to minimalny rozmiar obrazu, błyskawiczny start kontenera oraz wysokie bezpieczeństwo (brak kodu źródłowego wewnątrz działającego kontenera).

3.3. Izolacja i trwałość danych (Docker Volumes)

Kontenery aplikacyjne są ze swojej natury ulotne (*ephemeral*) – usunięcie lub restart kontenera powoduje bezpowrotne skasowanie wszystkich plików zapisanych w jego warstwie tymczasowej. Aby zapobiec utracie danych zgromadzonych w bazie przy restartach lub aktualizacjach środowiska, w pliku konfiguracyjnym zdefiniowano nazwany wolumen `mysql_data`. Wolumen ten mapuje wewnętrzny katalog serwera MySQL (`/var/lib/mysql`) bezpośrednio na bezpieczną, trwałą przestrzeń dyskową komputera hosta, zapewniając pełną integralność i trwałość danych.

4. Instrukcja uruchomienia projektu

Aby uruchomić system na dowolnym komputerze, wymagane jest posiadanie zainstalowanego oprogramowania Docker Desktop.

Krok 1: Wyłączenie lokalnych serwerów i usług

Upewnij się, że lokalne programy typu XAMPP, WampServer, MAMP czy systemowe usługi MySQL są całkowicie wyłączone, aby zwolnić porty sieciowe.

Krok 2: Uruchomienie terminala i nawigacja

Otwórz terminal systemowy (PowerShell na Windows lub Terminal na macOS) i przejdź do katalogu głównego projektu (tam gdzie znajduje się plik docker-compose.yml)

Krok 3: Czyszczenie środowiska (opcjonalne, zalecane przy błędach)

Jeśli w systemie operacyjnym znajdują się stare, zawieszone lub niepoprawnie zatrzymane kontenery pochodzące z poprzednich uruchomień, zaleca się wykonanie procedury twardego czyszczenia pamięci podręcznej i wolumenów za pomocą komendy:

```
docker-compose down --volumes --remove-orphans
```

Krok 4: Kompilacja i uruchomienie kontenerów

Wpisz komendę, która wymusi pobranie obrazów, kompilację kodu C# i start całego systemu:

```
docker-compose up --build
```

Poczekaj, aż w oknie konsoli przestaną lecieć logi startowe, Entity Framework Core automatycznie zaaplikuje wszystkie zaległe migracje bazodanowe i pojawi się zielona ikona oraz komunikat sygnalizacyjny: 🐾 Baza danych gotowa.

Krok 5: Otwarcie aplikacji w przeglądarce internetowej

Otwórz dowolną przeglądarkę internetową i przejdź pod dedykowany adres internetowy (port 8000 został dobrany tak, aby nie powodować konfliktów z systemową usługą AirPlay na komputerach Mac): <http://localhost:8000>

Krok 6: Autoryzacja na konto administratora

W systemie zaimplementowano mechanizm automatycznego seedowania konta administratora przy pierwszym uruchomieniu bazy danych. Zaloguj się w prawym górnym rogu na dedykowane konto deweloperskie:

- **Login:** superadmin
- **Hasło:** admin123

5. Instrukcja wersjonowania i zarządzania kodem projektu

Kod źródłowy systemu (w tym warstwa bazodanowa, aplikacyjna oraz pliki konfiguracyjne środowiska Docker) jest zarządzany przy użyciu rozproszonego systemu kontroli wersji Git.

5.1. Podejście do kontroli wersji

Archiwizacja i kopia zapasowa: System Git został wykorzystany jako narzędzie do ciągłej integracji kodu oraz zabezpieczenia historii zmian w trakcie pracy dwuosobowego zespołu

deweloperskiego. Każdy etap tworzenia kontrolerów, modeli Fluent API oraz widoków był na bieżąco utrwalany w lokalnym repozytorium.

Wydanie bazowe (v1.0.0): Aktualny stan projektu przedstawiony w niniejszej dokumentacji stanowi oficjalne, stabilne wydanie bazowe systemu (oznaczone roboczo jako wersja **v1.0.0**), w którym zintegrowano i pomyślnie przetestowano wszystkie wymagane moduły funkcjonalne.

5.2. Procedura wersjonowania na potrzeby przyszłego rozwoju

W przypadku dalszego rozwoju oprogramowania lub wdrażania poprawek, zespół deweloperski przyjmuje następujące wytyczne:

Zasada niezmienności obrazów: Ponieważ środowisko jest w pełni skonteneryzowane, każda nowa modyfikacja kodu po zatwierdzeniu w systemie Git wymaga odświeżenia kontenerów poprzez wywołanie komendy **docker-compose up --build**.

Semantyczne oznaczanie poprawek: Kolejne modyfikacje kodu (np. usunięcie błędów lub dodanie nowych funkcji) powinny skutkować podniesieniem numeru wersji aplikacji zgodnie ze standardem *Semantic Versioning* (np. **v1.0.1** dla poprawek bezpieczeństwa, **v1.1.0** dla nowych funkcjonalności).